Documentazione Tecnica di Progetto

GiaLoSai

Gestione Logistica e Stoccaggio

Gruppo 11

Corso di Ingegneria del Software

Anno Accademico 2025–2026

`Versione v2.0.0`

Università Degli Studi Di Perugia — Dipartimento Di Matematica E Informatica

1 Introduzione

1.1 Scopo del Progetto

Il software ha l'obiettivo di gestire le dinamiche logistiche interne di un'azienda, ottimizzando la gestione del magazzino, della movimentazione delle merci in entrata/uscita e della documentazione contabile associata.

1.2 Obiettivi e Ambito (Scope)

L'implementazione è un sistema client–server destinato a una rete locale: il client TUI comunica con il backend HTTP, che accede a MySQL. Il progetto modella l'intero ciclo della merce, dalla registrazione dei prodotti fino a DDT, fatture, note di credito e resi.

1.3 Glossario

- **DDT**: Documento di Trasporto.
- **RBAC**: Role-Based Access Control (Gestione permessi basata sui ruoli).
- **TUI**: Text-based User Interface, interfaccia utente da terminale.
- **DTO**: Data Transfer Object, struttura usata per lo scambio HTTP.
- **API**: Application Programming Interface, interfaccia di programmazione che espone endpoint HTTP per la comunicazione tra client e server.
- **REST**: Representational State Transfer, stile architetturale per servizi web basato su risorse identificate da URI e operazioni HTTP (GET, POST, PUT, DELETE).
- **MVC**: Model-View-Controller, pattern architetturale che separa il modello dati (Model), la presentazione (View) e la logica di controllo (Controller). Nella nostra architettura la View appartiene al processo client TUI, mentre i controller del server sono controller HTTP.
- **ORM**: Object-Relational Mapping, tecnica che mappa oggetti del dominio su tabelle relazionali, evitando di scrivere direttamente SQL. In Go, GORM è un esempio di ORM; noi usiamo `database/sql` con query parametrizzate per mantenere il controllo sulla persistenza.
- **JWT**: JSON Web Token, standard per la trasmissione di informazioni autenticate in forma compatta e sicura tramite token firmati.
- **CRUD**: Create, Read, Update, Delete, le quattro operazioni fondamentali di gestione dei dati.
- **SQL**: Structured Query Language, linguaggio dichiarativo per interrogare e modificare database relazionali.
- **Repository**: componente che incapsula l'accesso alla persistenza.
- **Stock-out**: Condizione di esaurimento delle scorte di un prodotto.

2 Analisi dei Requisiti

2.1 Attori del Sistema

- **Magazziniere**: Gestisce il flusso fisico delle merci (carico/scarico), effettua la scansione dei codici a barre, consulta le giacenze e genera i DDT per la merce in uscita.
- **Contabile**: Visiona l'elenco e il dettaglio degli ordini, gestisce le fatture, le note di credito e modifica i listini dei prezzi di acquisto e vendita.
- **Admin**: Supervisione globale del sistema, gestione degli utenti, consultazione delle merci e dei prezzi, controllo delle fatture, gestione dei resi e generazione della reportistica avanzata dell'inventario.

2.2 Requisiti Funzionali (RF)

Table 1: Requisiti Funzionali

ID	Descrizione	Priorità
RF-01	Login degli utenti tramite username e password basato su ruoli (RBAC).	Alta
RF-02	Blocco automatico dell'account utente a seguito di 3 tentativi di login falliti (l'account admin è esente).	Alta
RF-03	Recupero delle credenziali tramite l'admin, che sblocca l'account e reimposta la password dell'utente.	Media
RF-04	Visualizzazione, inserimento e modifica dei dati anagrafici dei prodotti e delle categorie.	Alta
RF-05	Consultazione in tempo reale dell'elenco delle giacenze di magazzino.	Alta
RF-06	Registrazione di un nuovo Ordine di Entrata e aggiornamento automatico incrementale delle giacenze.	Alta
RF-07	Registrazione di un nuovo Ordine di Uscita e decremento automatico delle giacenze dei prodotti coinvolti.	Alta
RF-08	Generazione di un DDT per ogni Ordine in uscita.	Alta
RF-09	Passaggio automatico dello stato dell'Ordine di Uscita da "Confermato" a "Evaso" alla creazione del DDT.	Alta
RF-10	Notifica visiva immediata per i prodotti la cui giacenza scende sotto la soglia minima di sicurezza.	Media
RF-11	Consultazione e gestione dei resi della merce	Media
RF-12	Visualizzazione e filtraggio degli ordini ricevuti (evasi e non evasi) da parte del contabile.	Alta

RF-13	Generazione e conservazione di fatture di vendita e note di credito con numerazione progressiva annuale.	Alta
RF-14	Modifica manuale dei listini dei prezzi di vendita e di acquisto dei prodotti.	Media
RF-15	Integrazione software per l'acquisizione rapida dei codici a barre dei prodotti tramite lettore ottico.	Media
RF-16	Gestione degli utenti: creazione, modifica, eliminazione e visualizzazione degli account da parte dell'admin.	Alta
RF-17	Visualizzazione dei report di fatturazione, da parte dei contabili.	Bassa

2.3 Requisiti Non Funzionali (RNF)

Table 2: Requisiti Non Funzionali

ID	Categoria	Descrizione
RNF-01	Prestazioni	Usando un linguaggio di programazione compilato (Go), il sistema deve garantire tempi di risposta rapidi e un basso consumo di risorse.
RNF-03	Portabilità	Funzionamento garantito in modalità offline su qualsiasi sistema operativo desktop moderno.
RNF-04	Sicurezza	Dati archiviati interamente in locale senza alcuna esposizione o transito all'esterno della rete aziendale.
RNF-05	Archiviazione	Persistenza relazionale centralizzata in MySQL 8, avviabile con Docker Compose.
RNF-06	Usabilità	Interfaccia testuale (TUI) leggera, navigabile da terminale.
RNF-07	Sicurezza	Autenticazione e autorizzazione basate su JWT e RBAC, con crittografia hashing.

2.4 Analisi dei Casi d'Uso

Il sistema è documentato tramite diagrammi dei casi d'uso che mappano le interazioni tra i tre attori principali (magazziniere, contabile e admin) e le macro-funzionalità: ordini, documenti fiscali, resi, inventario, prodotti e prezzi, utenti e autenticazione. Le relazioni di inclusione (*include*) e di estensione (*extend*) modellano rispettivamente i passi obbligati e le varianti dei flussi. I flussi dettagliati delle operazioni principali sono descritti nei diagrammi di sequenza e di attività allegati nella cartella di progetto.

Nota: I diagrammi seguenti sono esportati dai sorgenti PlantUML in `documentazione/uml/` e descrivono il modello di dominio e i flussi di design allineati al codice. La disponibilità effettiva di ogni funzionalità resta determinata dalle rotte del router e dalla navigazione della TUI: i diagrammi sono una vista di design, non una garanzia di integrazione end-to-end.

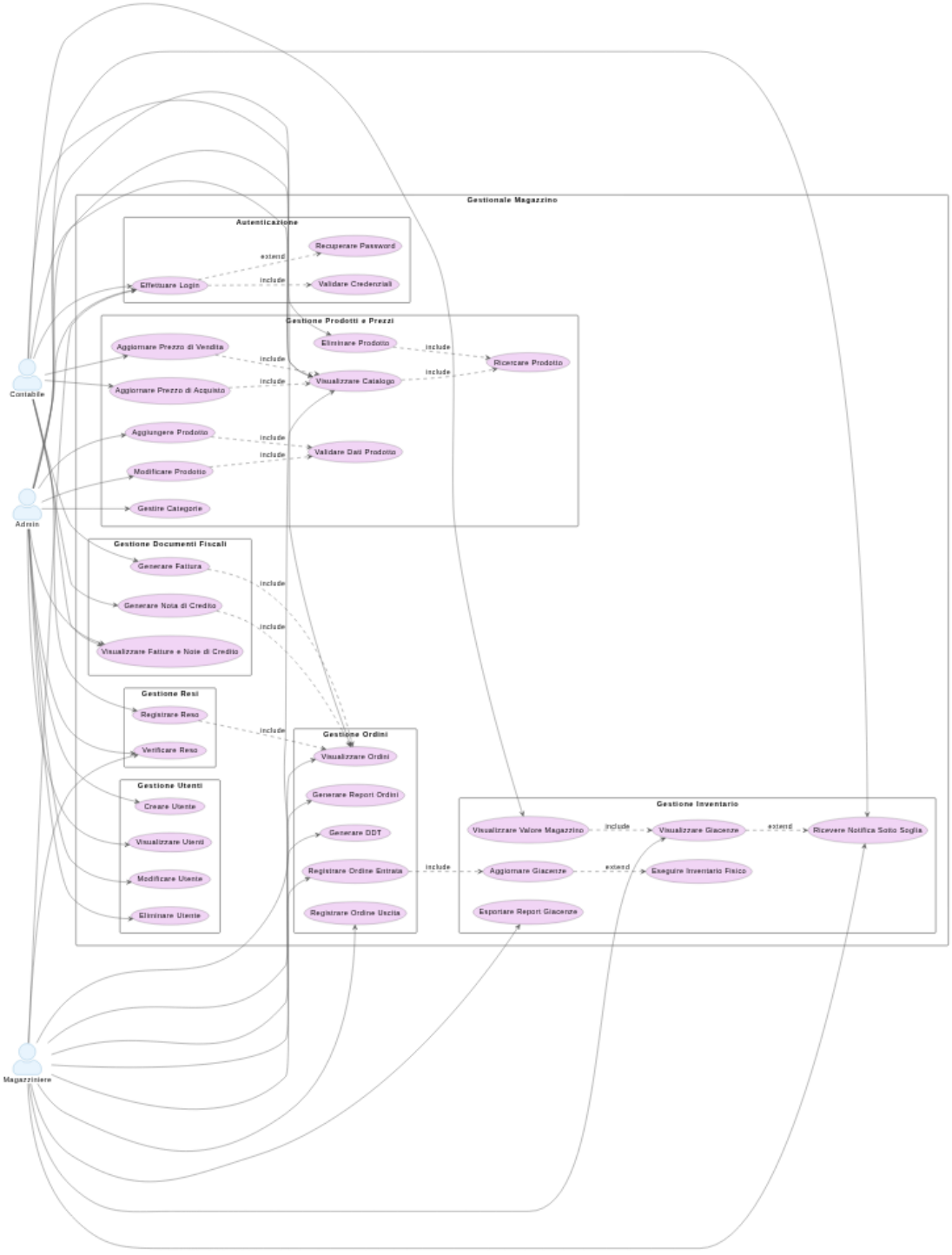


Figure 1: Diagramma dei Casi d'Uso Generale

3 Progettazione e Architettura

3.1 Scelte Architetturali

Il sistema combina gli stili *client–server*, *layered* e *repository*. Il client TUI è separato dal backend REST (Representational State Transfer) e comunica mediante HTTP (Hypertext Transfer Protocol) e JSON (JavaScript Object Notation). Nel server il router e il middleware costituiscono l'adattatore di trasporto; controller, service e repository separano rispettivamente conversione dei DTO, regole applicative e persistenza. Non si tratta di MVC (Model-View-Controller) classico: la View appartiene al processo client, mentre i controller del server sono controller HTTP.

> **Nota:** L'architettura client–server con TUI segue lo stesso pattern utilizzato da strumenti come `gh` (GitHub CLI) e `docker`: un client leggero che espone un'interfaccia testuale ricca, comunicando con un backend tramite API REST. Il pattern Repository è lo stesso adottato da `database/sql` in Go e da ORM (Object-Relational Mapping) come GORM, garantendo separazione tra logica applicativa e persistenza.

I repository incapsulano query MySQL parametrizzate. I service dipendono da interfacce ristrette e sono quindi verificabili con mock, coerentemente con Dependency Inversion. `TransactionHelper` propaga la transazione nel contesto, garantendo atomicità per operazioni quali aggiornamento delle giacenze e registrazione degli ordini. MySQL 8, InnoDB e le chiavi esterne sono configurati nello script `docker/mysql/init.sql`.

3.2 Modello del Dominio

Il dominio dell'applicazione è incentrato sulle entità fondamentali per il tracciamento logistico, mappate nel file `Domain_model.puml`: `Utente` e `Sessione`, `Categoria`, `Prodotto` e `Giacenza`, `OrdineEntrata` e `OrdineUscita` con le rispettive righe, `DDT`, `Fattura` (comprese le note di credito, distinte dal tipo) e `Reso`.

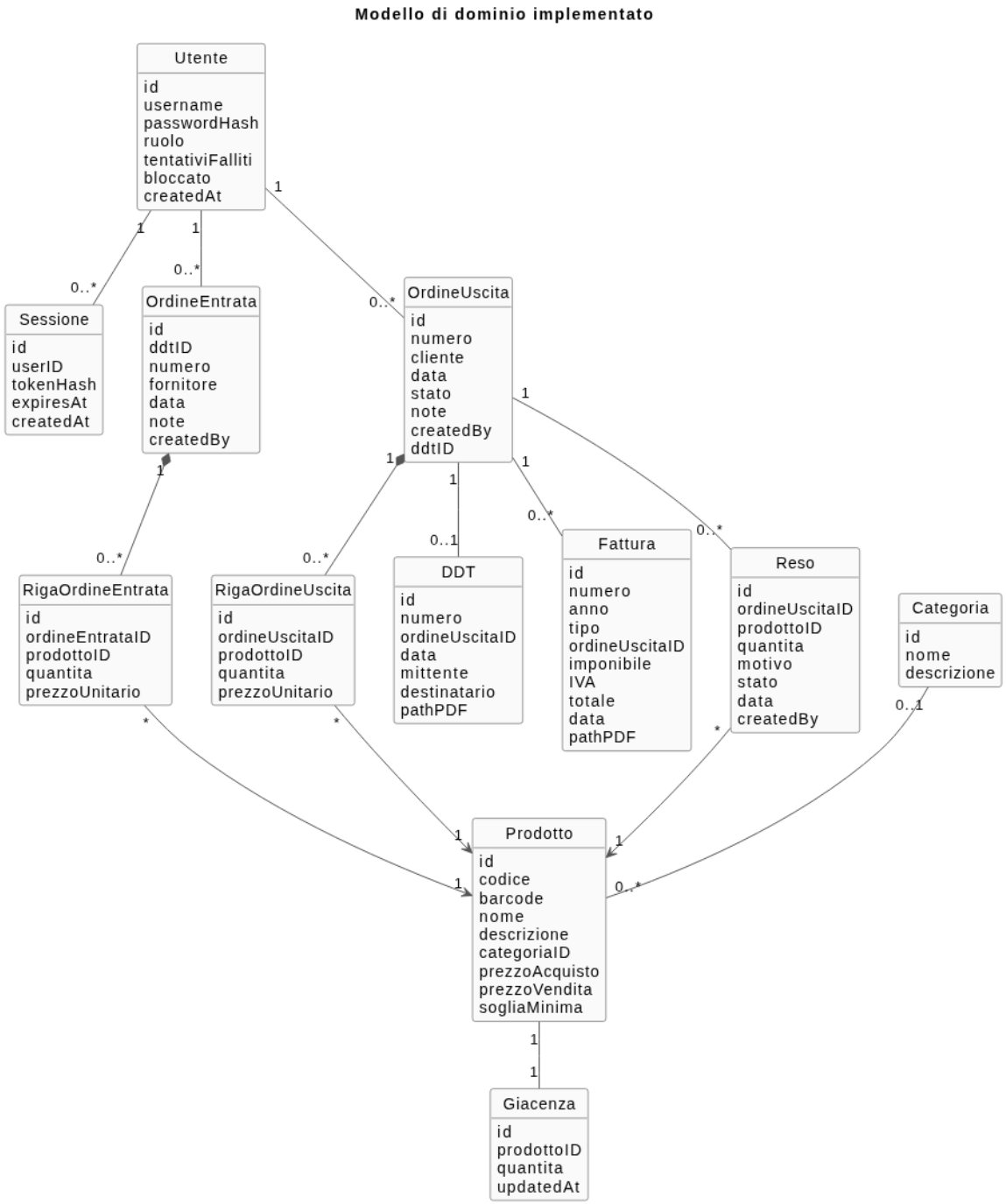


Figure 2: Modello del Dominio

3.3 Design Dettagliato (Diagramma delle Classi)

Il modello del codice comprende entità di dominio, DTO, controller, service e repository. Il pattern Repository isola il dialetto SQL e il dettaglio della connessione dal service layer. L'aggregazione dei componenti avviene in `router.NewRouter`; il dominio rimane indipendente da HTTP e da MySQL. Il diagramma delle classi è una vista di design e deve essere letto insieme al codice: le operazioni effettivamente pubblicate sono determinate dalle rotte del router.

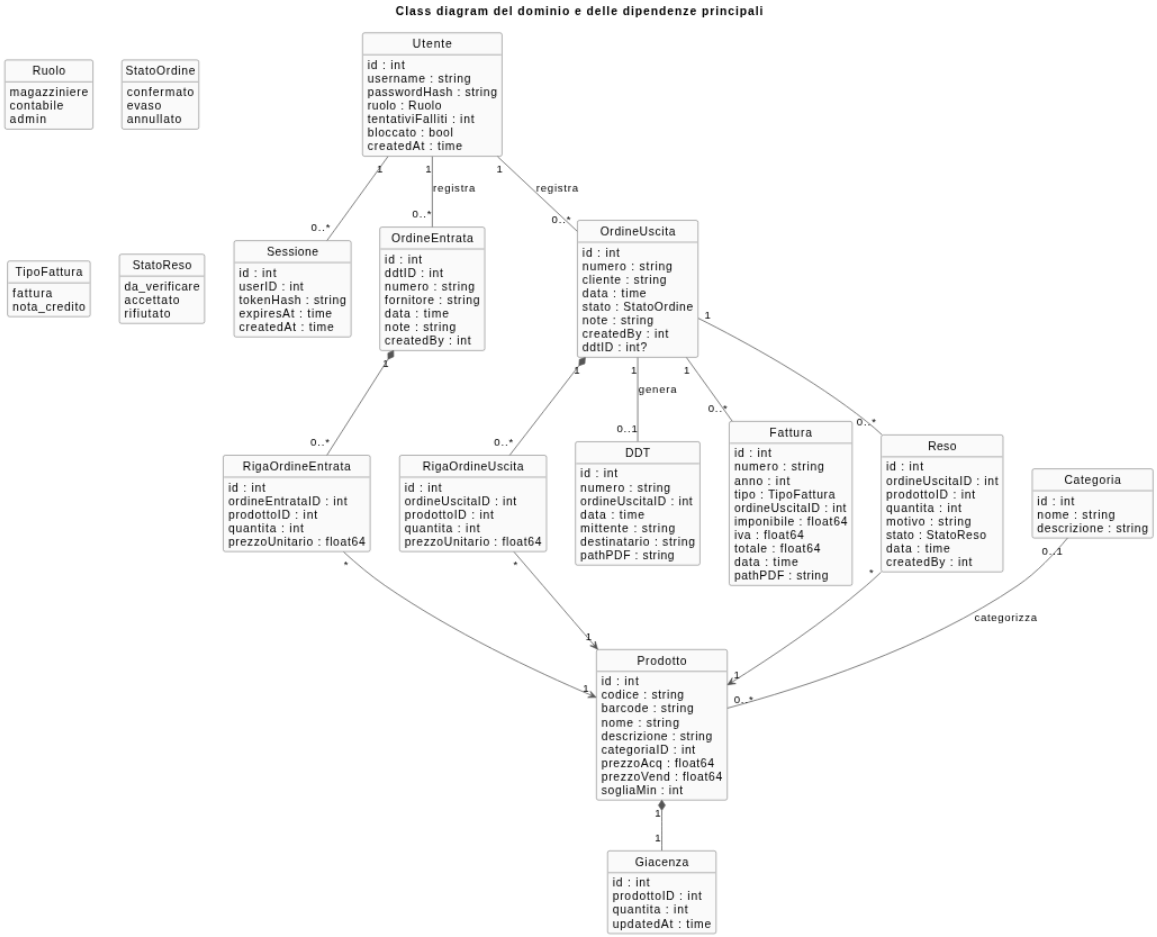


Figure 3: Diagramma delle Classi

3.4 Comportamento del Sistema (Diagrammi di Sequenza)

I diagrammi rappresentano scenari di requisiti e design, e descrivono i flussi implementati nel codice. Login, catalogo, categorie, prezzi, ordini (entrata/uscita), DDT, fatture, note di credito e resi sono tutti esposti come rotte HTTP dal router, e le relative operazioni sono raggiungibili dalla navigazione TUI. I diagrammi sono quindi una vista di design allineata al codice; la disponibilità effettiva di ogni operazione resta comunque determinata dalle rotte del router e dalla navigazione della TUI.

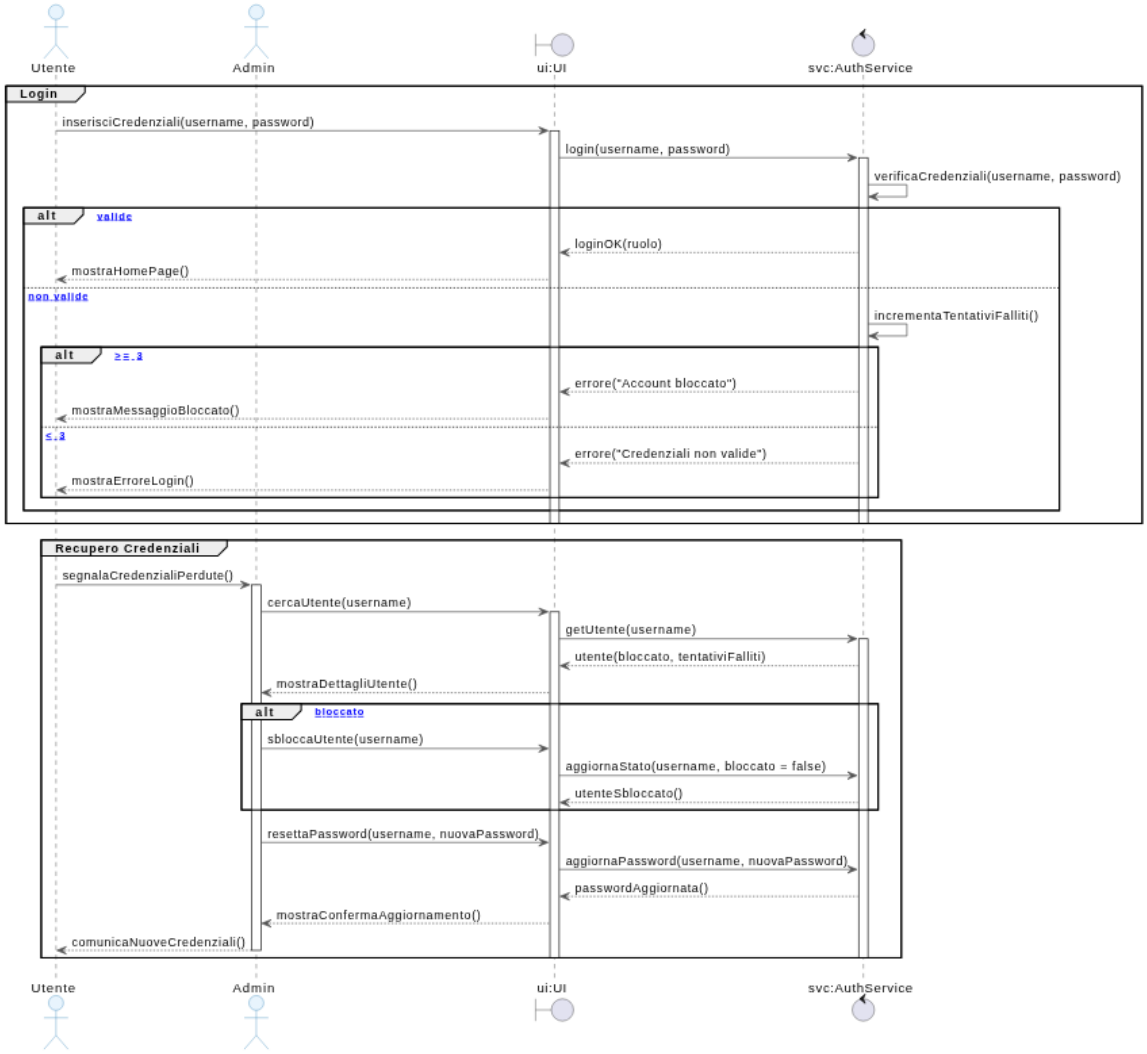


Figure 4: Diagramma di Sequenza: Login e Recupero Password (UC1)

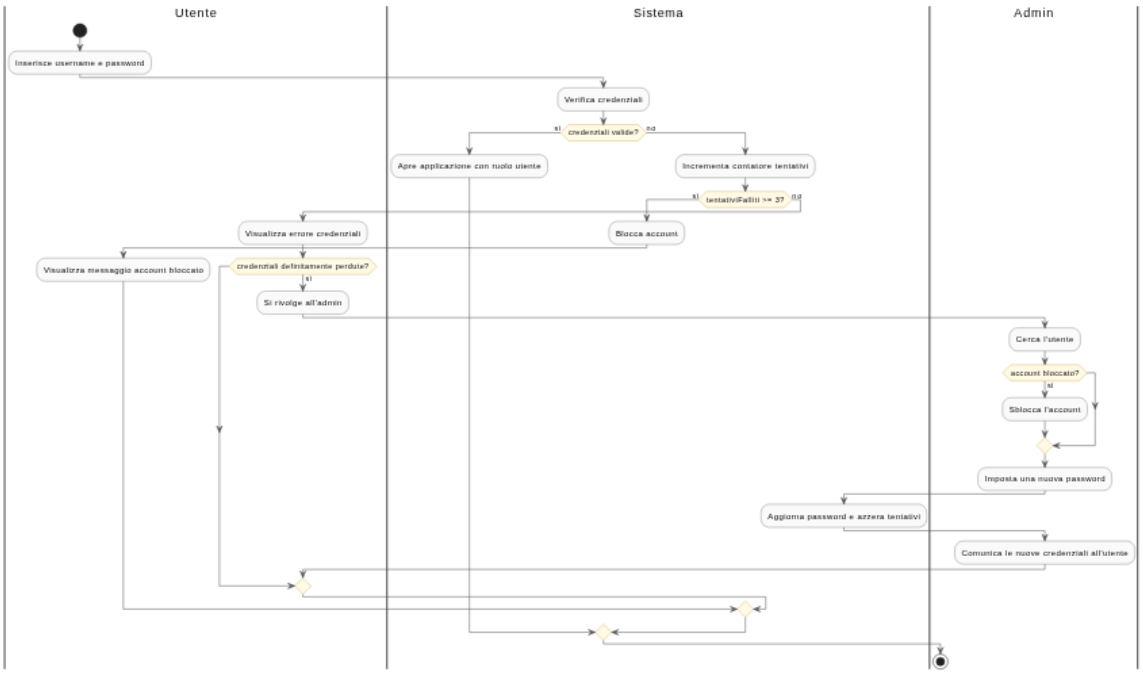


Figure 5: Diagramma di Attività: Login e Recupero Password (UC1)

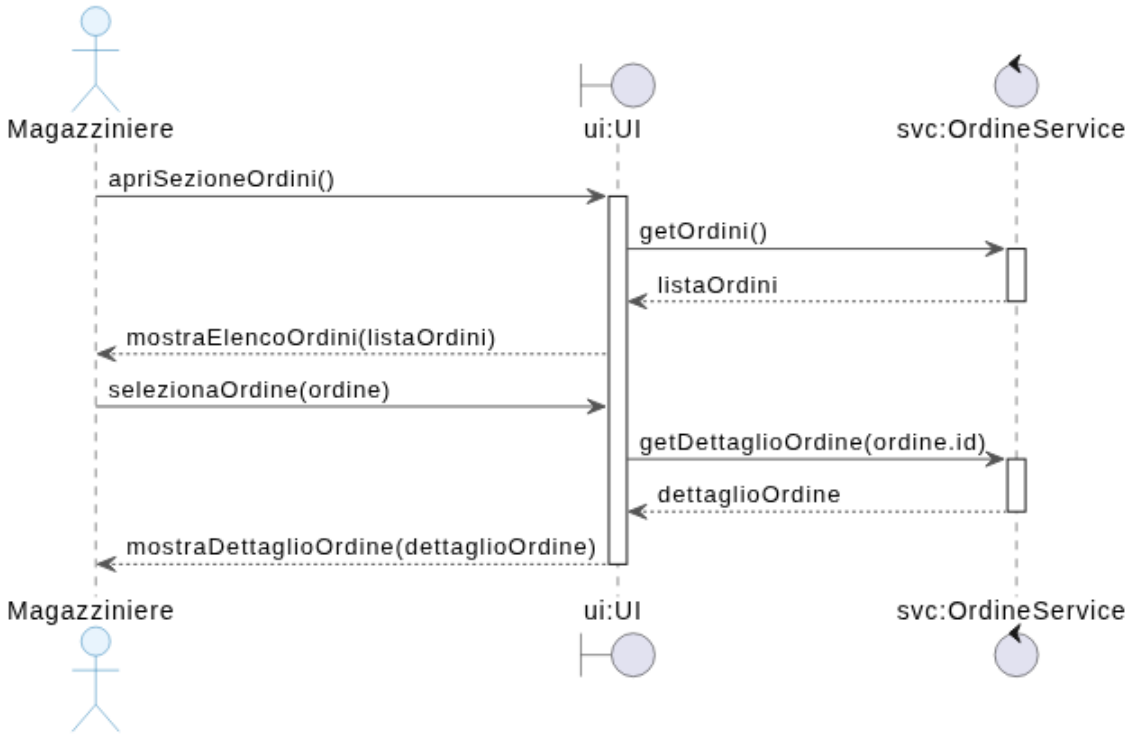


Figure 6: Diagramma di Sequenza: Visualizzazione Ordini (UC2)

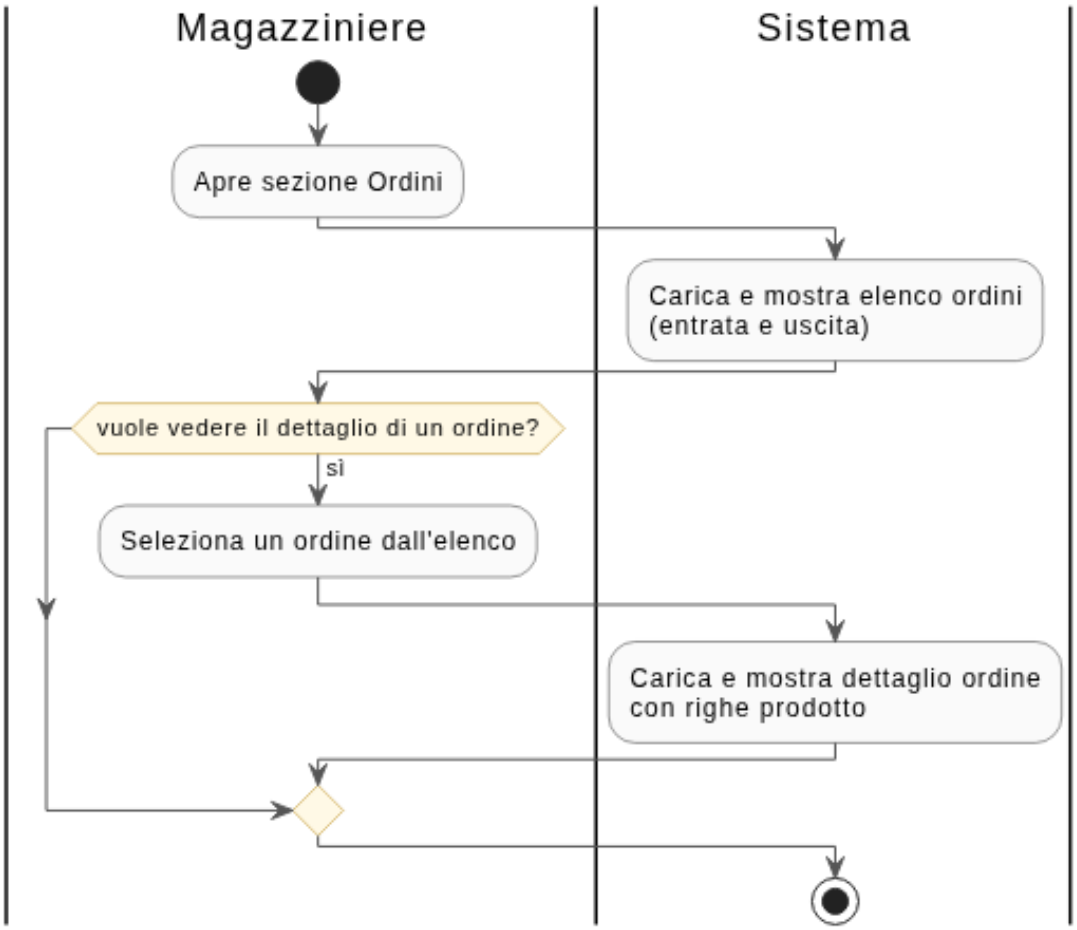


Figure 7: Diagramma di Attività: Visualizzazione Ordini (UC2)

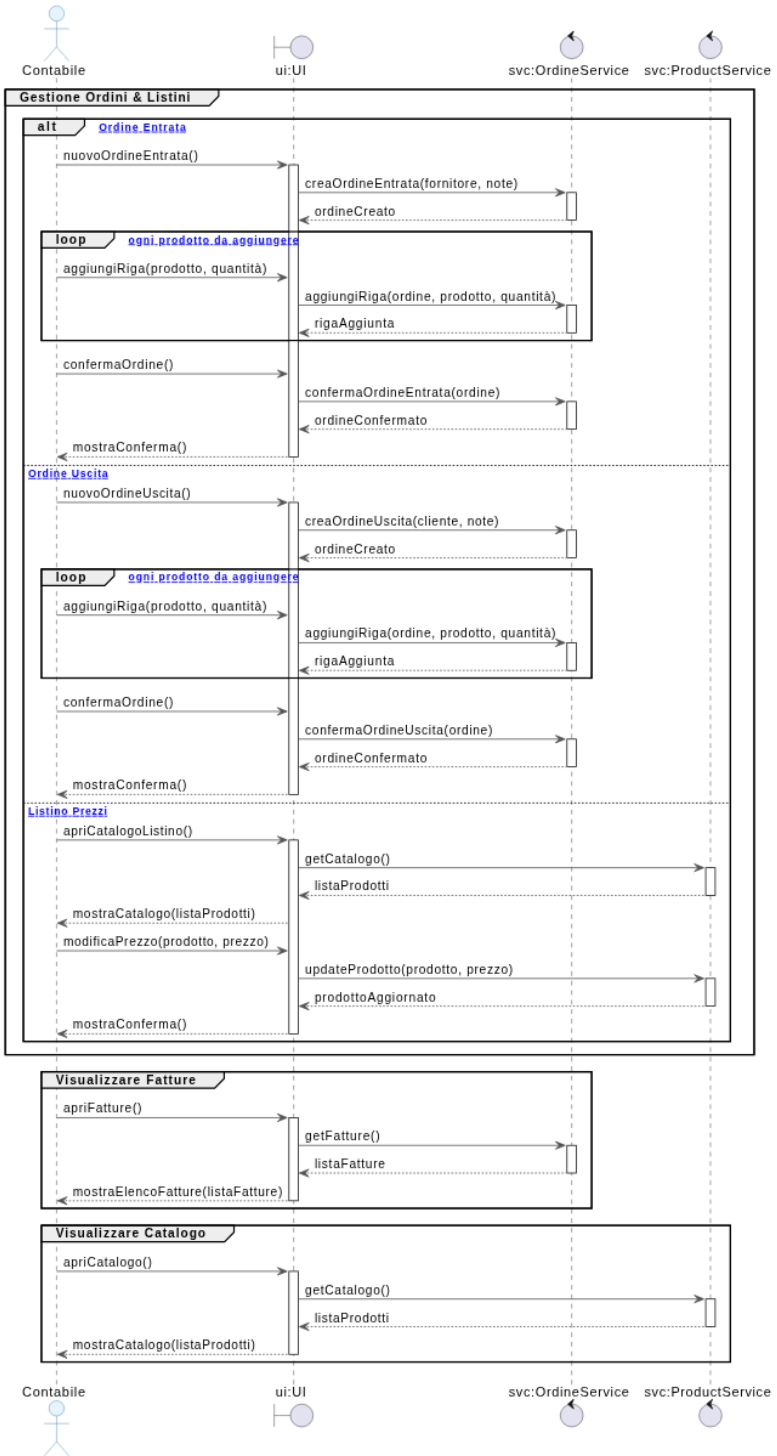


Figure 8: Diagramma di Sequenza: Gestione Ordini & Listini (UC3)

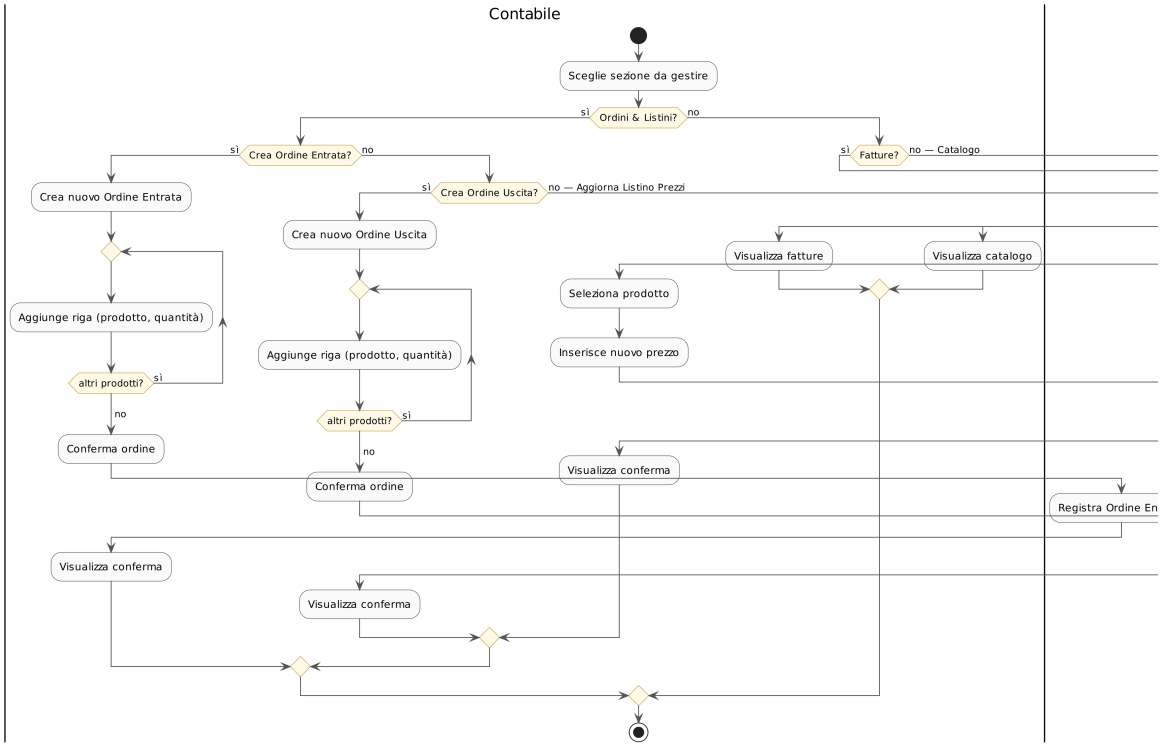


Figure 9: Diagramma di Attività: Gestione Ordini & Listini (UC3)

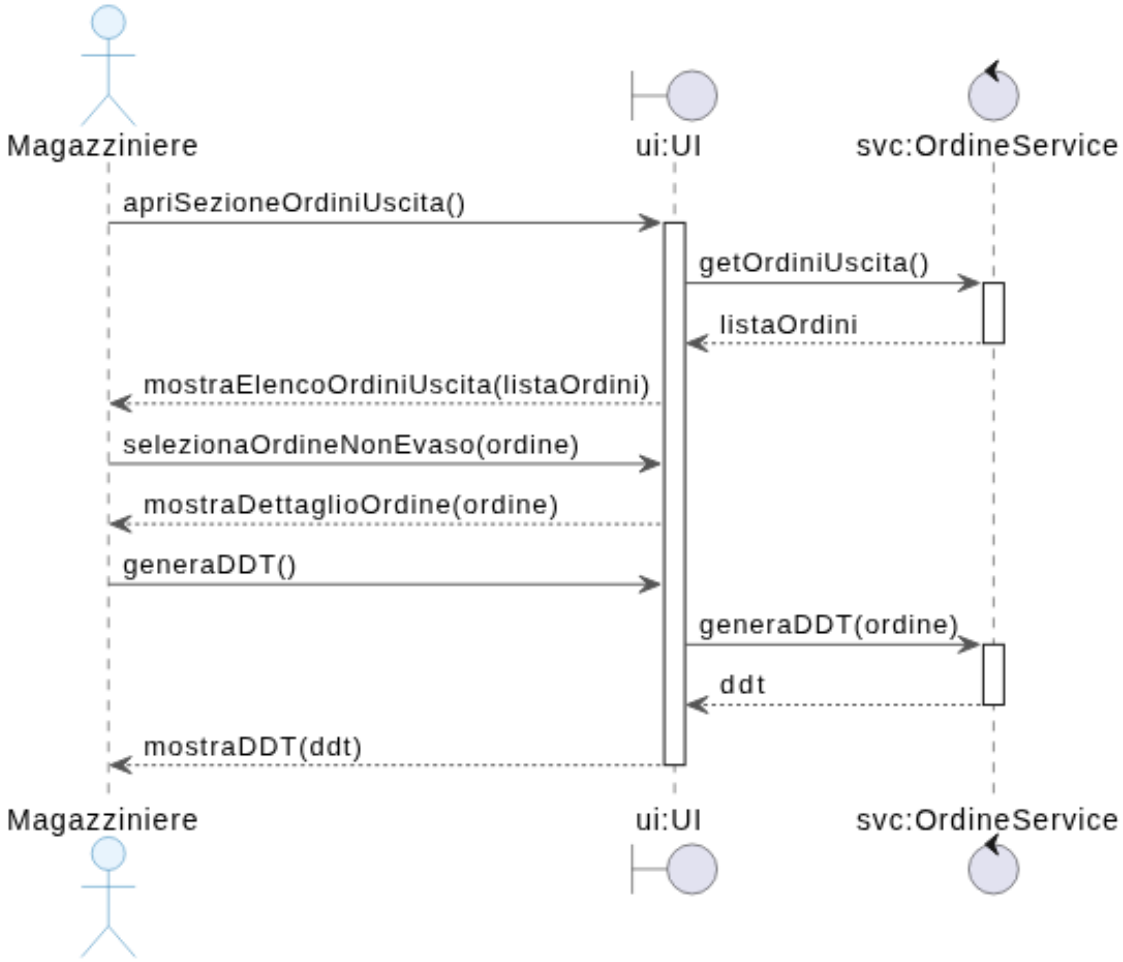


Figure 10: Diagramma di Sequenza: Generazione DDT (UC4)



Figure 11: Diagramma di Attività: Generazione DDT (UC4)

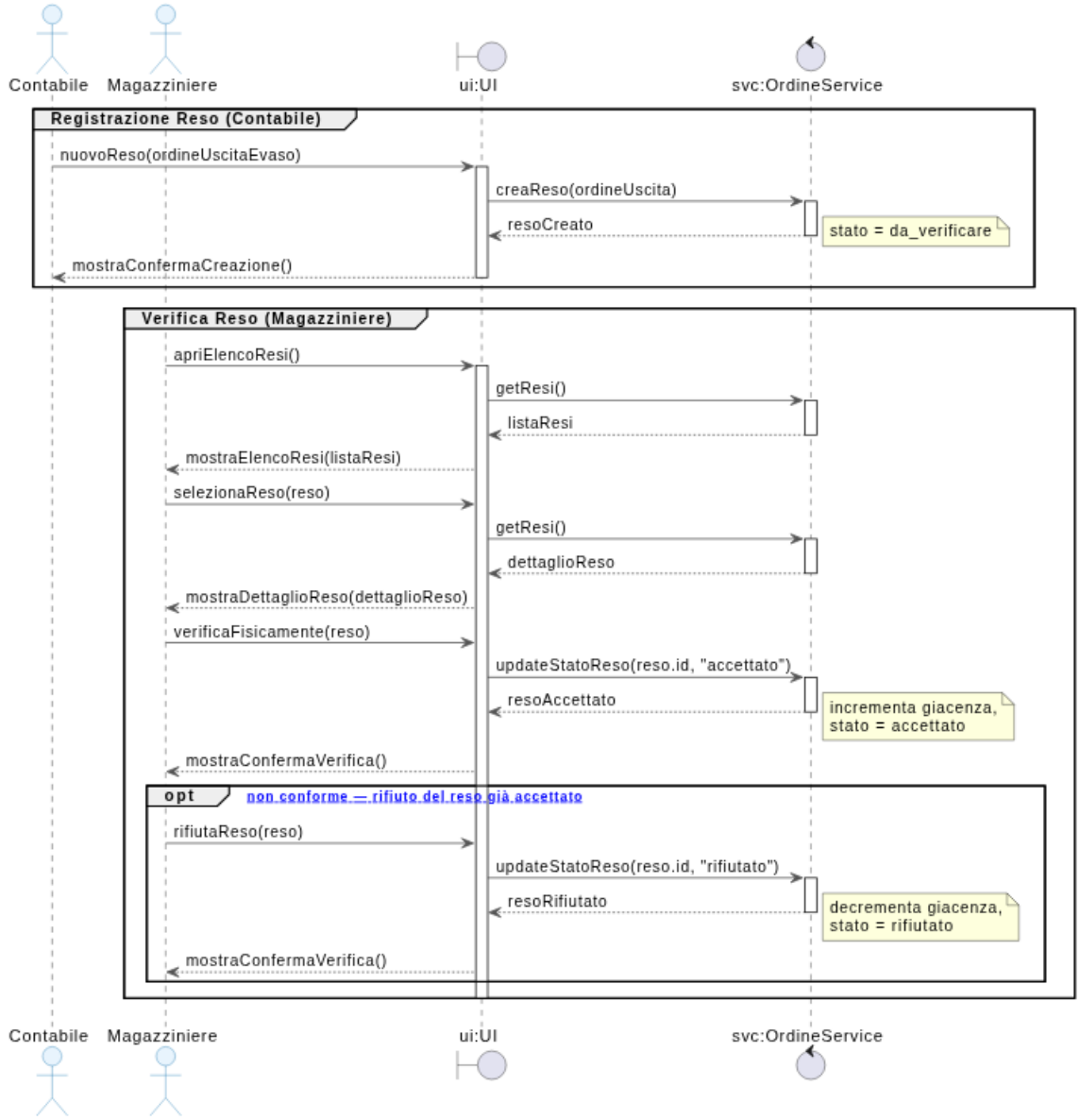


Figure 12: Diagramma di Sequenza: Resi (UC5)

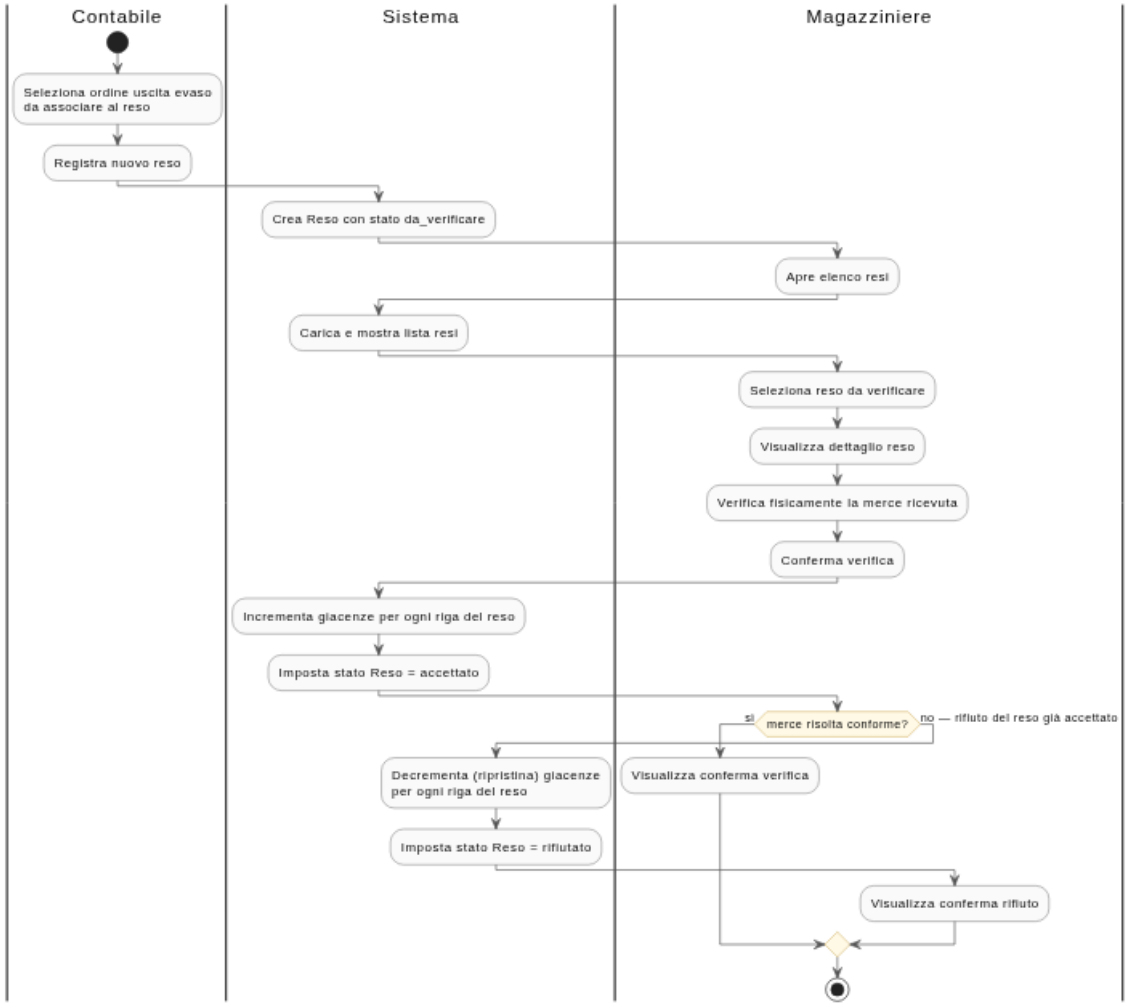


Figure 13: Diagramma di Attività: Resi (UC5)



Figure 14: Diagramma di Sequenza: Contabile Ordini (UC6)



Figure 15: Diagramma di Attività: Contabile Ordini (UC6)

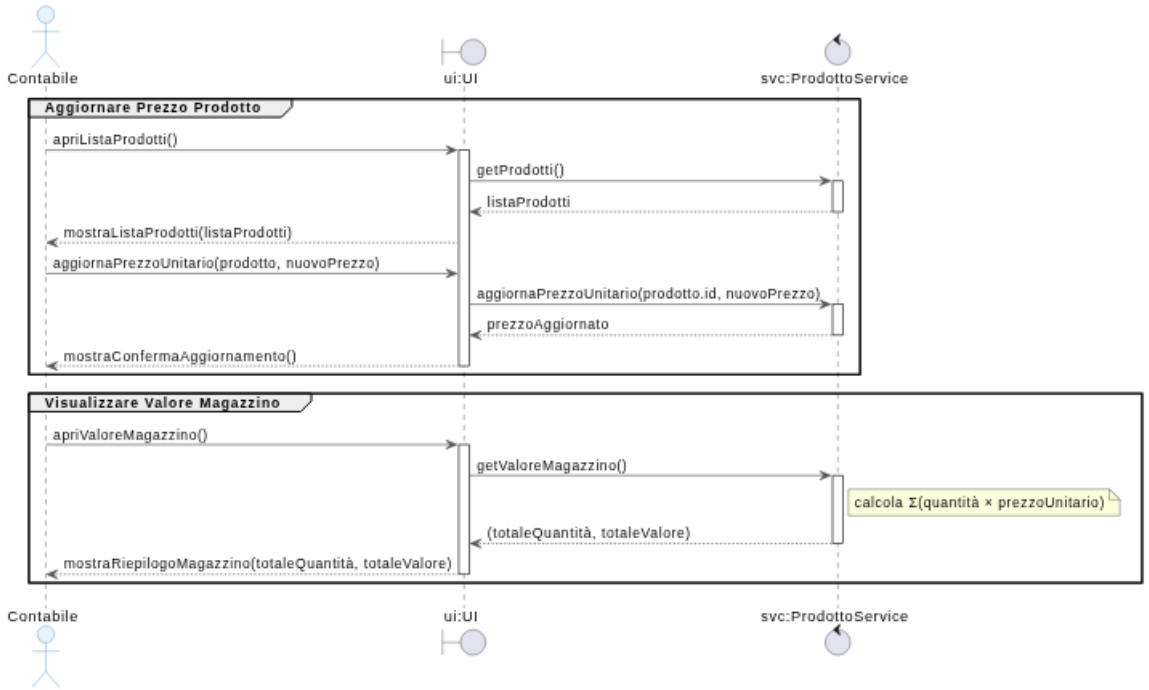


Figure 16: Diagramma di Sequenza: Contabile Prezzi (UC7)

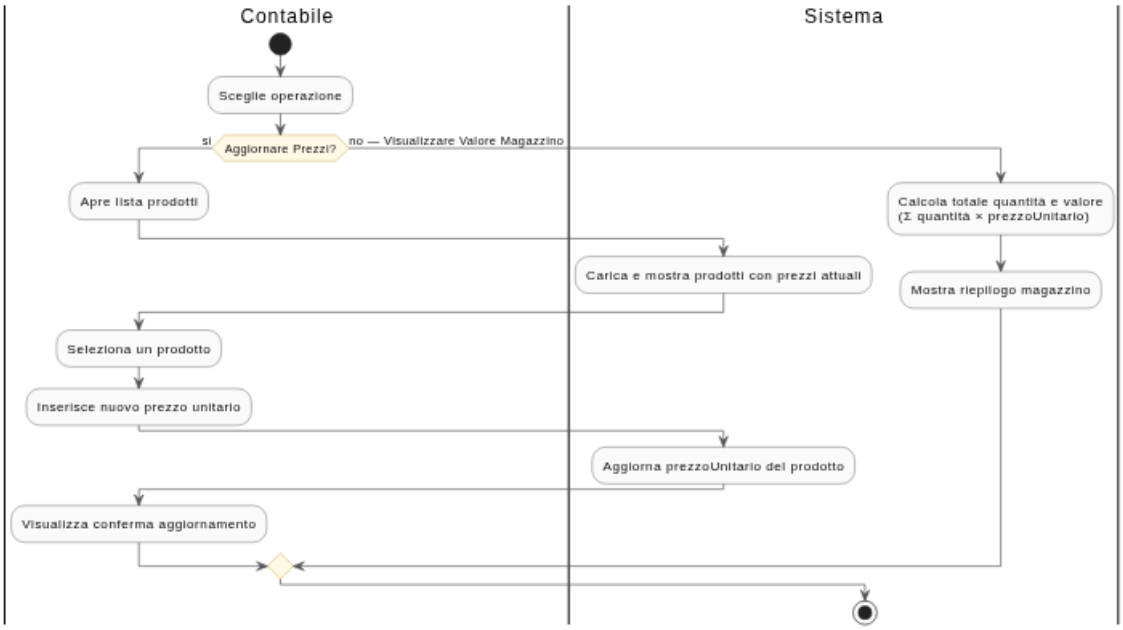


Figure 17: Diagramma di Attività: Contabile Prezzi (UC7)

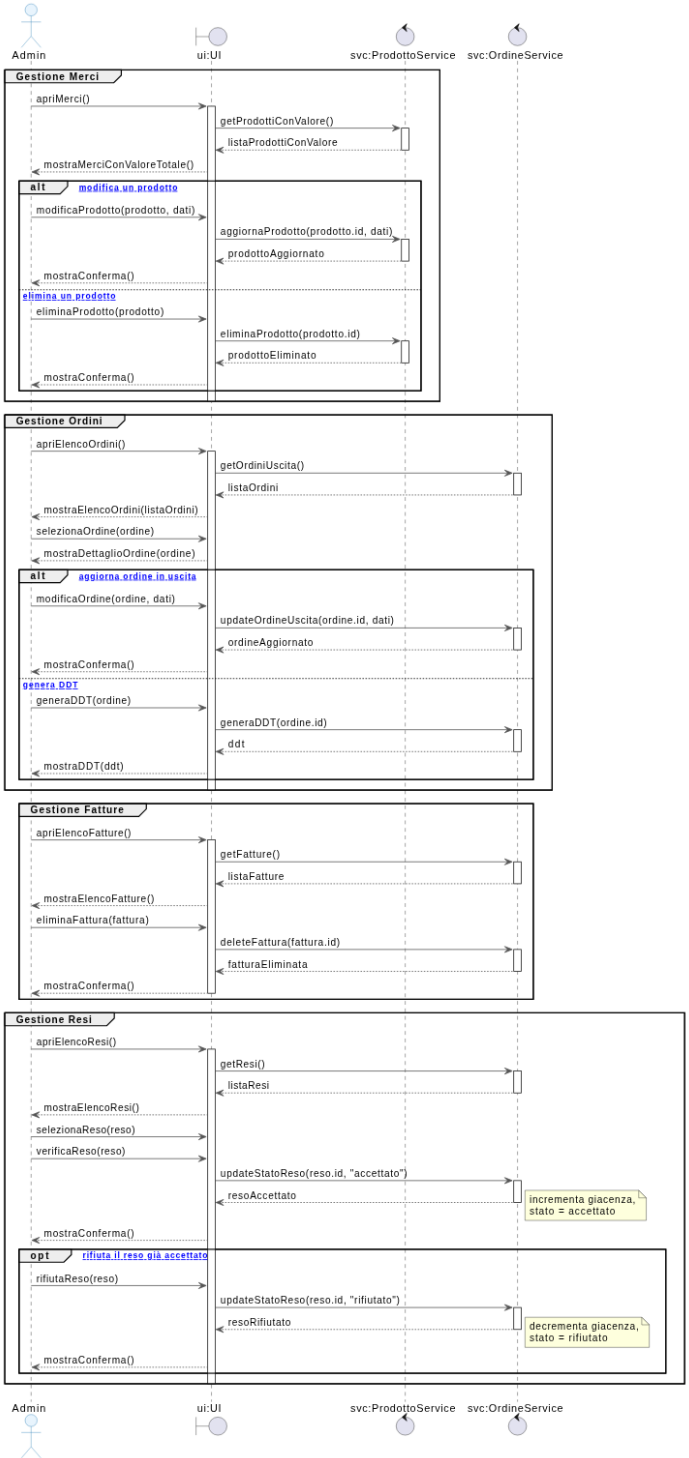


Figure 18: Diagramma di Sequenza: Admin (UC8)

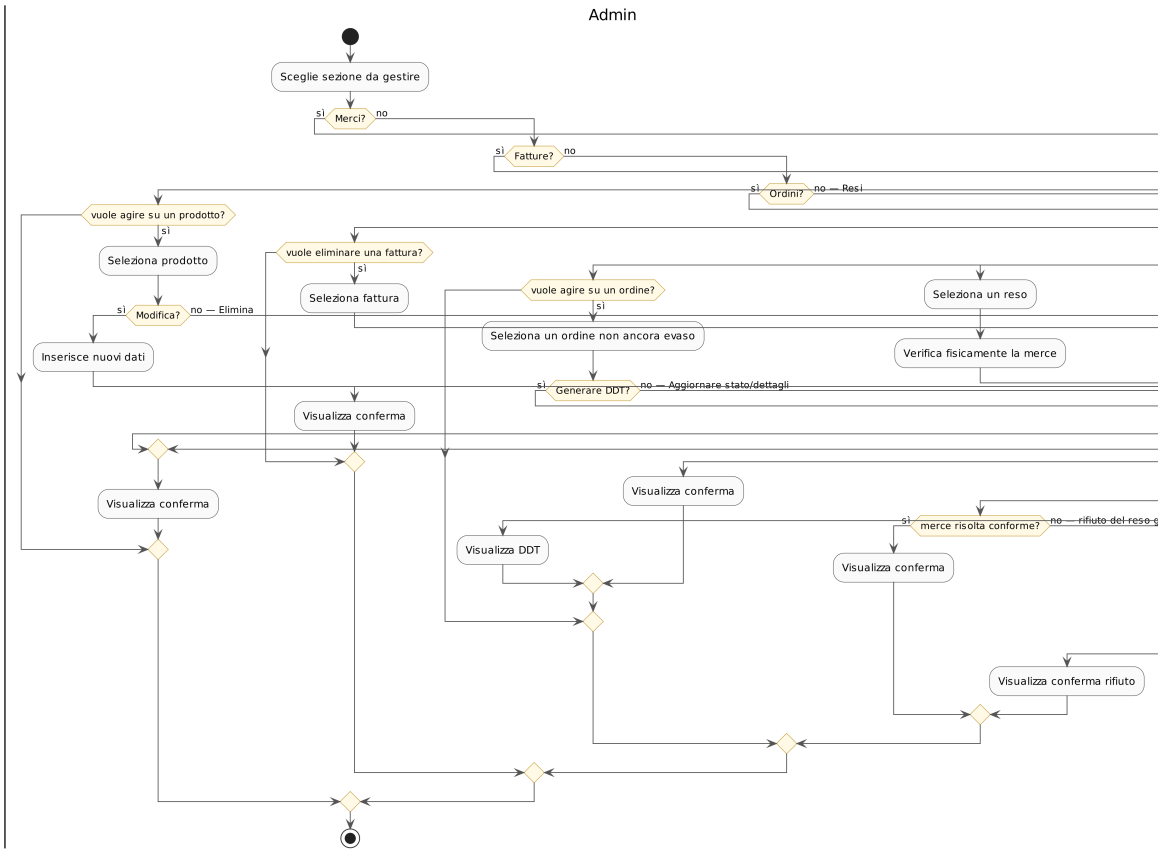


Figure 19: Diagramma di Attività: Admin (UC8)

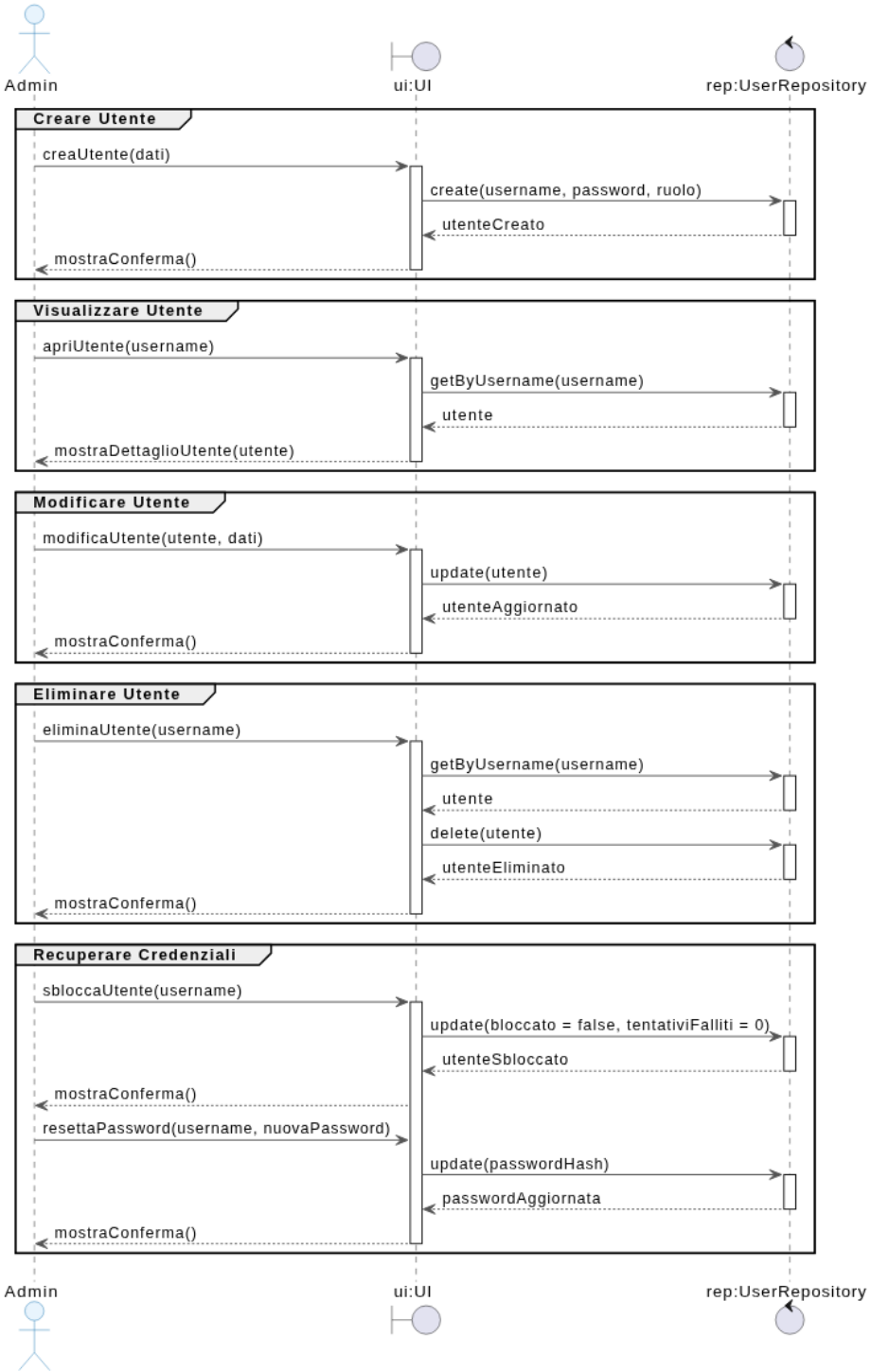


Figure 20: Diagramma di Sequenza: Gestione Utenti (UC9)

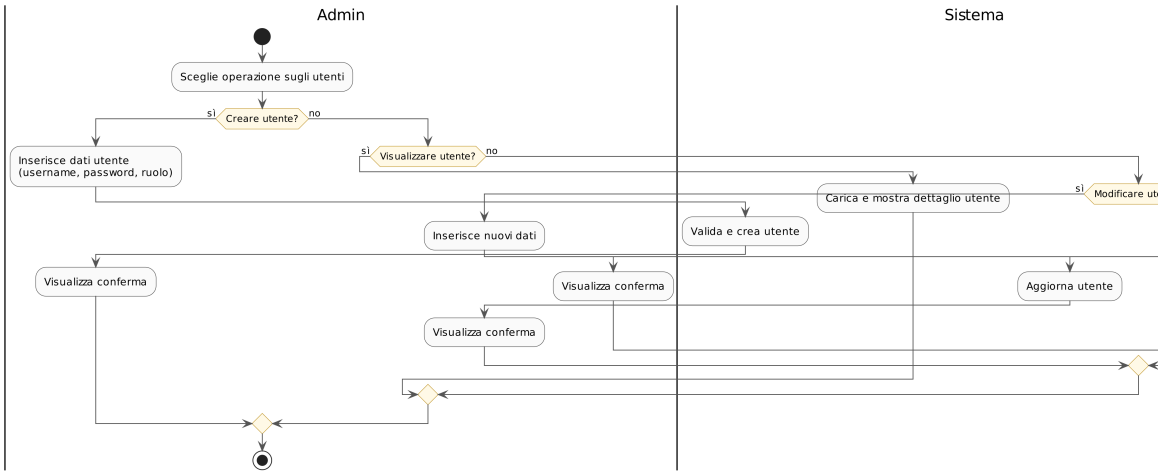


Figure 21: Diagramma di Attività: Gestione Utenti (UC9)

4 Gestione del Progetto (Scrum)

4.1 Backlog

L'elenco ordinato e prioritizzato dei requisiti tecnici necessari al completamento del primo ciclo di sviluppo comprende:

1. Configurazione dell'architettura software di base, dell'ambiente Docker e della connessione a MySQL.
2. Definizione dello schema logico della base dati tramite script SQL (creazione delle tabelle `Utente`, `Prodotto`, `Categoria`, `Giacenza`, `OrdineEntrata`, `OrdineUscita`, `RigaOrdineEntrata`, `RigaOrdineUscita`, `DDT`).
3. Sviluppo delle interfacce grafiche e delle funzioni CRUD (Create, Read, Update, Delete) per la gestione dell'Anagrafica Prodotti e Categorie.
4. Implementazione del modulo di movimentazione (logica e maschere per la compilazione di `OrdineEntrata` e `OrdineUscita`).
5. Integrazione software per l'acquisizione dell'input da lettore di codici a barre tramite emulazione tastiera.
6. Sviluppo del sottosistema di controllo e notifica visiva per i prodotti sotto la soglia minima inseriti in `Giacenza`.
7. Sviluppo del modulo di autenticazione locale e gestione dei permessi basata sui ruoli tramite `AuthService`.

4.2 User Stories

Le storie riflettono i bisogni concreti degli attori scritti secondo gli standard del framework Scrum:

- **US-01 (Accesso con credenziali)**: "Come utente, voglio accedere all'applicazione con username e password, in modo da poter utilizzare le funzionalità del mio ruolo".
- **US-02 (Scansione Barcode)**: "Come magazziniere, voglio poter scansionare il codice a barre di un prodotto in arrivo tramite il lettore ottico, per aggiornare la quantità in giacenza nel sistema in modo rapido, automatizzato e senza commettere errori manuali".
- **US-03 (Sicurezza Account)**: "Come admin del sistema, voglio che il software blocchi l'account dopo 3 tentativi falliti, per impedire ad utenti non autorizzati di indovinare le password d'accesso dei dipendenti".
- **US-04 (Recupero Credenziali)**: "Come admin, voglio poter sbloccare eventuali utenti bloccati, per consentire loro di accedere nuovamente all'applicazione".
- **US-05 (Visualizzazione elenco ordini)**: "Come magazziniere, voglio vedere l'elenco di tutti gli ordini di entrata e uscita, in modo da avere una panoramica completa dei movimenti del magazzino".
- **US-06 (Visualizzazione dettaglio ordini)**: "Come magazziniere, voglio vedere il dettaglio di un ordine di entrata o uscita selezionato, in modo da conoscere i prodotti coinvolti e le relative quantità".

- **US-07 (Registrazione Ordini in entrata)**: "Come contabile, voglio registrare un ordine di entrata con i prodotti ricevuti, in modo da aggiornare automaticamente le giacenze".
- **US-08 (Registrazione Ordini in uscita)**: "Come contabile, voglio registrare un ordine di uscita con i prodotti richiesti, in modo da aggiornare automaticamente le giacenze".
- **US-09 (Generazione DDT)**: "Come magazziniere, voglio generare un documento di trasporto (DDT) per ogni ordine di uscita evaso, in modo da modificarne lo stato eavere una documentazione ufficiale per la merce in uscita".
- **US-10 (Visualizzare Giacenze)**: "Come magazziniere, voglio visualizzare in tempo reale le giacenze di magazzino, in modo da sapere sempre la quantità disponibile di ogni prodotto".
- **US-11 (Creazione resi)**: "Come contabile, voglio generare le pratiche di reso della merce, in modo da poter operare correttamente i resi".
- **US-12 (Visualizzazione ordini per contabile)**: "Come contabile, voglio visualizzare l'elenco degli ordini ricevuti (evasi e non evasi), in modo da poter gestire la fatturazione e le note di credito".
- **US-13 (Generazione Fatture)**: "Come contabile, voglio generare fatture per gli ordini di uscita evasi, in modo da poter emettere documenti ufficiali per i clienti".
- **US-14 (Nota di credito)**: "Come contabile, voglio generare una nota di credito per un ordine di uscita evaso, in modo da correggere la fatturazione quando necessario".
- **US-15 (Gestione prezzi)**: "Come contabile, voglio poter modificare manualmente i listini dei prezzi di vendita e di acquisto dei prodotti, in modo da mantenere aggiornati i prezzi nel sistema".
- **US-16 (Gestione utenti)**: "Come admin, voglio creare, modificare ed eliminare gli account degli utenti, in modo da gestire l'accesso al sistema e i relativi ruoli".
- **US-17 (Reportistica inventario)**: "Come contabile, voglio visualizzare report avanzati dell'inventario, in modo da avere una visione dettagliata dello stato del magazzino e delle tendenze di consumo dei prodotti".
- **US-18 (Modifica stato reso)**: "Come magazziniere, voglio poter modificare lo stato di un reso, in modo da aggiornare correttamente le giacence e la documentazione associata".
- **US-19 (Accesso privilegiato)**: "Come admin, voglio poter avere accesso alle funzioni riservate agli altri tipi di utenti, in modo da poter supervisionare e gestire l'intero sistema".

Nota: Per le fatture e le note di credito (RF-13) l'importo mostrato è calcolato a partire dalle righe dell'ordine come *totale*, cioè imponibile più IVA al **22%**: $\texttt{totale} = \texttt{imponibile} + (\texttt{imponibile} \times 0.22)$, con $\texttt{imponibile} = \sum_{riga}(\texttt{quantita} \times \texttt{prezzo_unitario})$. Il valore visualizzato nella TUI è quindi *prezzo + 22% di IVA*.

4.3 Criteri di Accettazione

Vengono definiti i vincoli formali affinché ciascuna User Story possa essere considerata completata ("Done"). I criteri sono organizzati per area funzionale, a partire dalle principali storie del progetto.

Autenticazione e Sicurezza (US-01, US-03, US-04)

- **US-01 – Accesso con credenziali**: il sistema deve accettare il login solo con username e password validi; in caso di successo deve restituire un token JWT e il ruolo dell'utente, e deve inoltrare l'utente alla home del proprio ruolo.
- **US-03 – Sicurezza Account**: dopo 3 tentativi di login falliti consecutivi l'account deve essere bloccato automaticamente, con l'eccezione dell'account `admin`; un account bloccato non deve poter autenticarsi.
- **US-04 – Recupero Credenziali**: l'admin deve poter sbloccare un account bloccato (con azzeramento dei tentativi falliti) e reimpostarne la password; il ripristino deve essere registrato nel database.

Prodotti e Giacenze (US-02, US-10)

- **US-02 – Scansione Barcode**: il sistema deve riconoscere e decodificare correttamente e senza latenza i codici a barre conformi allo standard internazionale EAN-13. Al momento della scansione riuscita all'interno della schermata di carico, i dati identificativi del prodotto devono comparire istantaneamente. Qualora il codice non trovi corrispondenza in `Prodotti`, il service restituisce un errore di prodotto non trovato. Feedback acustico, pop-up e inserimento contestuale non sono implementati nella TUI corrente.
- **US-10 – Visualizzare Giacenze**: l'elenco delle giacenze deve mostrare per ogni prodotto la quantità disponibile e segnalare visivamente i prodotti sotto la soglia minima di sicurezza, aggiornando i valori in tempo reale.

Ordini (US-05, US-06, US-07, US-08)

- **US-05/06 – Visualizzare ordini**: il sistema deve mostrare l'elenco degli ordini di entrata e uscita e consentire di visualizzare il dettaglio di un ordine selezionato, con le righe prodotto e le relative quantità.
- **US-07/08 – Registrare ordini**: la registrazione di un ordine di entrata deve incrementare le giacenze dei prodotti coinvolti; la registrazione di un ordine di uscita deve verificare la disponibilità e decrementare le giacenze. Se la quantità richiesta supera la giacenza disponibile, la transazione deve annullarsi e restituire un errore di giacenza insufficiente.

Documenti e Contabilità (US-09, US-12, US-13, US-14, US-15)

- **US-09 – Generazione DDT**: la generazione di un DDT deve essere possibile solo per ordini di uscita confermati e deve portare automaticamente lo stato dell'ordine a "Evaso".
- **US-12 – Ordini per il contabile**: il contabile deve poter visualizzare gli ordini di uscita, distinguendo quelli evasi da quelli non evasi.

- **US-13/14 – Fatture e note di credito**: la generazione deve essere possibile solo per ordini di uscita nello stato "Evaso" e deve calcolare correttamente imponibile, IVA (22%) e totale, assegnando una numerazione progressiva annuale.
- **US-15 – Gestione prezzi**: la modifica del prezzo di vendita o di acquisto di un prodotto deve essere persistita nel database e riflessa nelle viste successive del catalogo.

Resi, Utenti e Reportistica (US-11, US-16, US-17)

- **US-11 – Gestione resi**: la registrazione di un reso deve creare il reso con stato "da_verificare"; alla verifica, lo stato deve passare a "accettato" o "rifiutato".
- **US-16 – Gestione utenti**: l'admin deve poter creare, visualizzare, modificare ed eliminare gli account utente; il sistema deve impedire la cancellazione di account con ordini a esso associati.
- **US-17 – Reportistica inventario**: il report deve includere per ogni prodotto giacenza, soglia minima e stato di sotto-soglia, restituito in formato JSON dal router.

5 Strategia di Verifica e Validazione

5.1 Piano di Testing

Il piano prevede una suite di test di unità (Unit Testing) sistematici eseguiti sulla Business Logic tramite lo strumento standard di testing di Go (`go test`). I test sono focalizzati sulla verifica delle transazioni e sul rispetto dei vincoli di integrità del magazzino.

Oltre ai test di unità, la cartella `server/router/` contiene una suite di *test di integrazione end-to-end* (`integration_test.go`, 17 casi) che attraversano l'intera catena HTTP → router → middleware → controller → service → repository, con database mockato tramite `go-sqlmock`. Questi test verificano il contratto tra frontend e backend: path e metodo HTTP, status code e forma del body JSON, esattamente come il client TUI li attende quando comunica con il server reale.

5.2 Casi di Test (Test Cases)

TC-01: Tentativo di scarico merce con quantità richiesta superiore alla giacenza fisica attuale.

- **Scenario**: Un prodotto registrato in anagrafica dispone di una quantità fisica in `Giacenza` pari a 5 unità. Viene inserito un `OrdineUscita` con una `RigaOrdineUscita` richiedente l'uscita di 6 unità del medesimo prodotto.
- **Input**: Invocazione della funzione di business `registraScarico()` con quantità = 6.
- **Risultato Atteso**: Il service intercetta l'incoerenza prima di modificare i dati, la transazione MySQL esegue il `ROLLBACK` e viene restituito un `GiacenzaInsufficienteError`.

Il service impedisce quindi giacenze negative; la visualizzazione dell'errore dipende dall'integrazione TUI/API della specifica operazione.